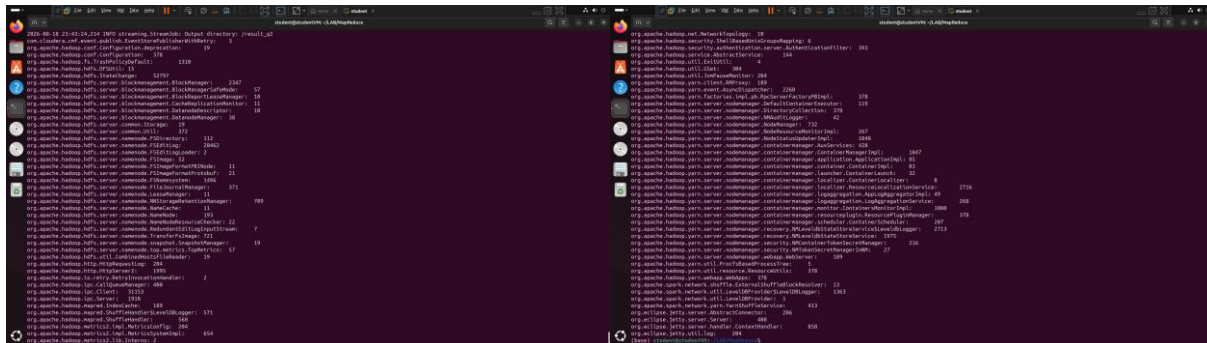


LAB 2: MapReduce

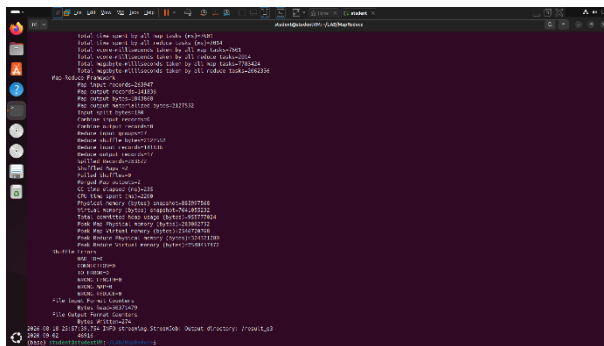
1. How many logs are there in each log level (INFO, WARN, ERROR)?

```
(base) student@studentVM:~/LAB/MapReduce$ hadoop fs -cat /result_q1/part-00000
ERROR      362
INFO      140224
WARN      1250
(base) student@studentVM:~/LAB/MapReduce$
```

2. How many logs are there in each type of source?



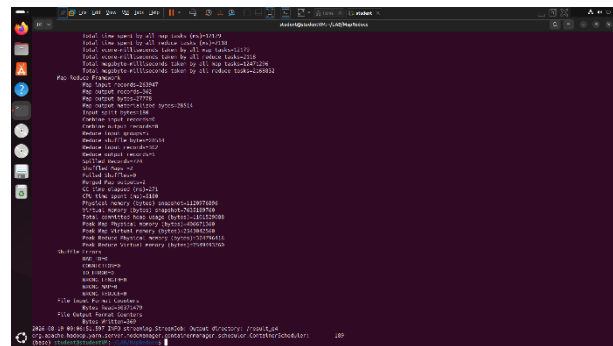
3. Which day has highest number of logs occurred and how many?



- Day with highest number of logs: 2020-09-02
- Total log count: 46,916

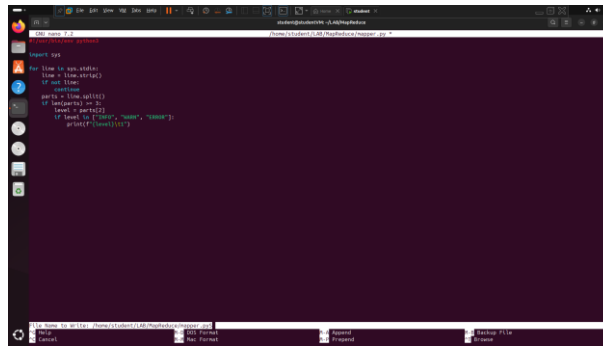
4. Which source has highest number of ERROR level logs occurred?

- **Source with highest ERROR logs:**
org.apache.hadoop.yarn.server.nodemanager.containermanager.scheduler.ContainerScheduler
- **Total ERROR log count: 189**



Mapper.py

1. First Code Edit



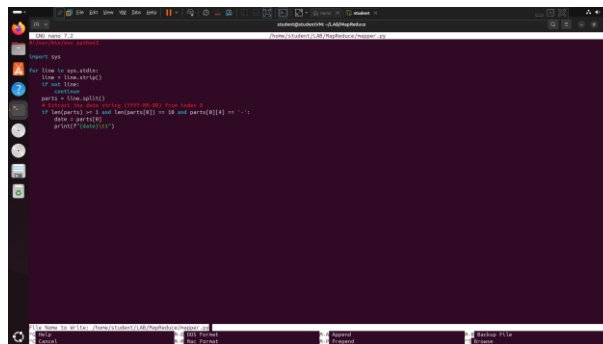
- `#!/usr/bin/env python3`: Tells Linux to run this script using Python 3.
- `import sys`: Imports Python's system module so the script can read standard input (`sys.stdin`).
- `for line in sys.stdin::` Loops through the incoming log file line by line as Hadoop streams data into the mapper.
- `line = line.strip()`: Removes extra trailing spaces and newline characters (`\n`) from the start and end of the line.
- `if not line: continue`: Skips blank or empty lines to avoid processing errors.
- `parts = line.split()`: Splits the log line into a list of words/tokens based on space boundaries.
- `if len(parts) >= 3::` Ensures the line has at least 3 fields before accessing array indexes.
- `level = parts[2]`: Extracts index 2 (the 3rd word in the line), which holds the log level (INFO, WARN, ERROR).
- `if level in ["INFO", "WARN", "ERROR"]::` Filters out unexpected tokens and verifies the value is a valid log level.
- `print(f"{level}\t1")`: Outputs the key-value pair formatted as LOG_LEVEL + tab character (`\t`) + 1 (e.g., INFO\t1).

2. Second Code Edit



- `#!/usr/bin/env python3`: Shebang line that tells Linux to run the file using Python 3.
- `import sys`: Imports the system library so the script can read input directly from the terminal or Hadoop stream (`sys.stdin`).
- `for line in sys.stdin::` Iterates through incoming log entries line by line as Hadoop passes data to the mapper.
- `line = line.strip()`: Strips trailing spaces, tabs, and newline characters (`\n`) from the ends of each line.
- `if not line: continue`: Skips blank lines so empty data isn't processed.
- `parts = line.split()`: Splits the log string into individual words/tokens wherever space occurs.
- `if len(parts) >= 4::` Ensures the log line has at least 4 fields so the script won't crash on incomplete lines.
- `source = parts[3]`: Grabs index 3 (the 4th element), which contains the Java package/class name (e.g., `org.apache.hadoop.yarn...`).
- `print(f"{source}\t1")`: Emits the package name followed by a tab (`\t`) and 1, handing off key-value pairs to Hadoop's reducer.

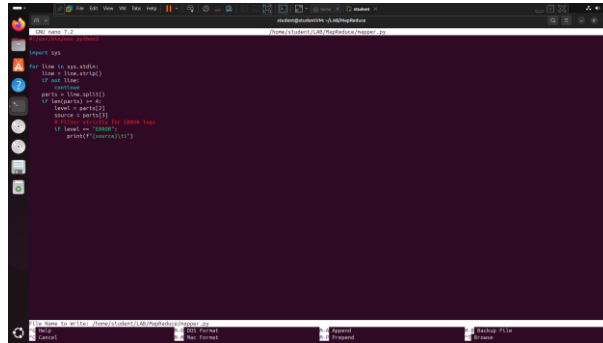
3. Third Code Fix



- `parts[0]`: Extracts the first column of the log line, which contains the date format (e.g., `2026-08-18`).
- `len(parts[0]) == 10 and parts[0][4] == '-'`: Filters out invalid lines or stack trace headers that don't start with a standard date string.

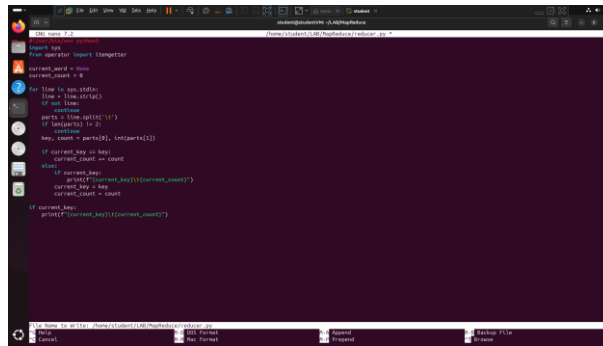
- `sort -k2 -nr | head -n 1`: Sorts the final reducer counts numerically (`-n`) in reverse order (`-r`) based on column 2 (`-k2`), then prints only the top result (`head -n 1`).

4. Code update 4th time



- `parts = line.split()`: Splits the log string into individual words/tokens based on space boundaries.
- `if len(parts) >= 4`: Ensures the log line has at least 4 fields (date, time, log level, source class) to prevent index errors.
- `level = parts[2]`: Extracts index 2 (the 3rd token), which contains the log severity level (e.g., INFO, WARN, ERROR).
- `source = parts[3]`: Extracts index 3 (the 4th token), which contains the Java class/source package name.
- `if level == "ERROR"`: Filters the log entries to ensure only lines with an ERROR status are processed.
- `print(f"{source}\t1")`: Outputs the source package name followed by a tab character (`\t`) and 1, handing off the filtered key-value pairs to Hadoop's reducer to sum up total errors per source.

Reducer.py



- `current_key = None & current_count = 0`: Initializes tracking variables to remember the log level currently being counted.
- `for line in sys.stdin::` Reads sorted key-value pairs (`LOG_LEVEL\t1`) generated by the mapper and sorted by Hadoop.
- `parts = line.split('\t')`: Splits the incoming line into key and count using the tab separator.
- `key, count = parts[0], int(parts[1])`: Assigns the log level string to key and converts the string "1" into an integer 1.
- `if current_key == key::` Checks if the incoming log level matches the one currently being aggregated. If yes, it adds `count` to `current_count`.
- `else::` Triggered when the log level changes (e.g., transitioning from `ERROR` lines to `INFO` lines):
 - `if current_key::` Prints the total count for the *previous* log level before switching.
 - `current_key = key & current_count = count`: Resets the tracker for the new log level.
- `if current_key: (final lines)`: Outputs the accumulated count for the last remaining log level after reading all lines.